

Pensamento Lógico Computacional com Python

Aula 01

Prof.ª Larissa

Bibliografia

- PERKOVIC, L. *Introdução à computação usando Python: um foco no desenvolvimento de aplicações*. Rio de Janeiro: LTC, 2022.
- BARRY, P. *Use a cabeça! Python*. Rio de Janeiro: Alta Books, 2021.
- LAMBERT, K. A. *Fundamentos de Python: Primeiros programas*. São Paulo: Cengage Learning, 2023.
- MUELLER, J. P. *Começando a programar em Python para leigos*. [s.l.]: Alta Books, 2020.
- BANIN, S. L. *Python 3 - conceitos e aplicações - uma abordagem didática*. São Paulo: Erica, 2018.
- DOWNEY, A. B. *Pense em Python*. Disponível em: <https://pense-python.caravela.club/introducao.html>. Acesso em: 09 fev. 2025.
- PYTHON SOFTWARE FOUNDATION. *Python 3.14.3 documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 21 fev. 2026.

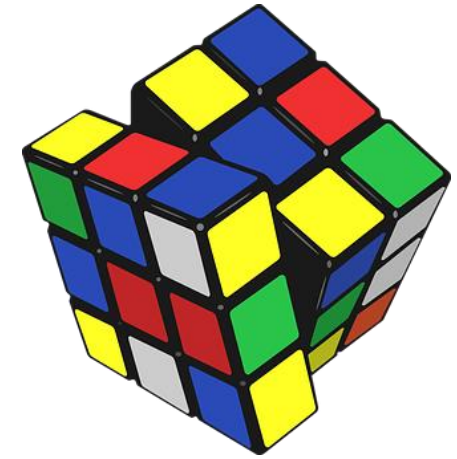
Sumário

- Introdução à lógica de programação
 - Algoritmo x programa
 - Simbologia de fluxogramas
 - Exemplos de fluxogramas
- Introdução à linguagem Python
 - Sobre a linguagem
 - Compilador x interpretador
 - Modos de programação
 - Operadores aritméticos
 - Operadores relacionais
 - Operador de atribuição
 - Comandos de entrada e de saída
 - Comentários

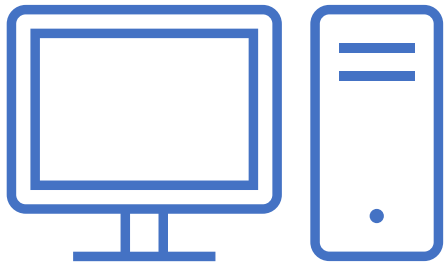
Introdução à lógica de programação: algoritmo x programa

- Algoritmo

Um **algoritmo** pode ser definido como uma sequência de instruções destinada à realização de uma tarefa. Pense em uma “receita de bolo”. Pode ser executado por um humano.



- Programa



No contexto da computação, um **programa** pode ser definido como um algoritmo implementado por uma linguagem de programação. O algoritmo é escrito em um código e é executado por processadores digitais.

Introdução à lógica de programação: fluxogramas

Um fluxograma é um diagrama que descreve visualmente, por meio de formas geométricas, um algoritmo. Muito aplicado a programas computacionais.

- Simbologia básica de um fluxograma



Início ou fim do programa.



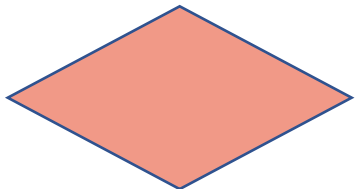
Declaração de variáveis: as variáveis são usadas para armazenar dados em memória durante a execução do programa.



Atribuição: expressa qualquer processamento a ser executado no conteúdo de uma variável.



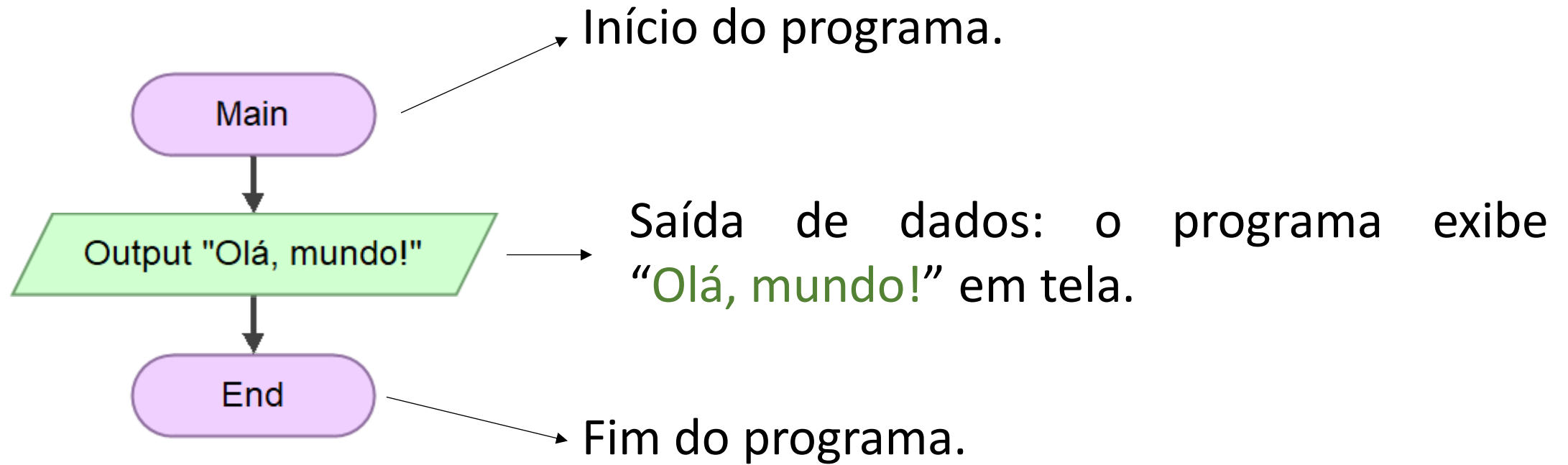
Entrada/saída de dados: lê um valor de entrada de dados e armazena em uma variável, ou mostra um valor em tela.



Decisão: verifica uma expressão lógica e executa um desvio, baseado no resultado verdadeiro ou falso.

Introdução à lógica de programação: exemplos de fluxogramas

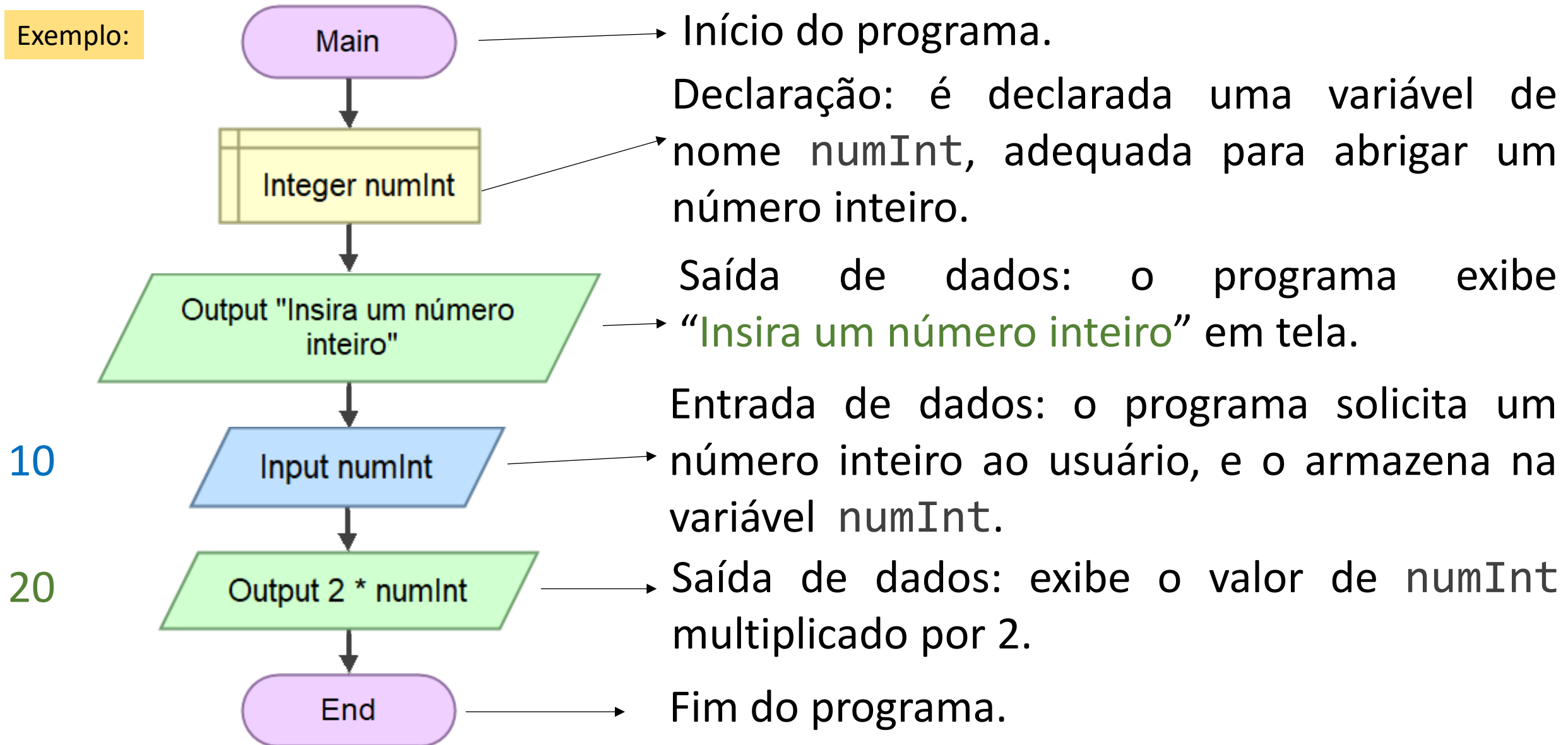
Fluxograma de um programa que exibe “Olá, mundo!”.



Introdução à lógica de programação: exemplos de fluxogramas

Fluxograma de um programa que solicita um nº inteiro e exibe o dobro desse nº.

Exemplo:



Introdução à lógica de programação: exemplos de fluxogramas

Programa que calcula o consumo médio de combustível de um veículo, em km/L.

$$\text{Consumo} = \frac{\text{distância percorrida [km]}}{\text{combustível gasto [L]}}$$

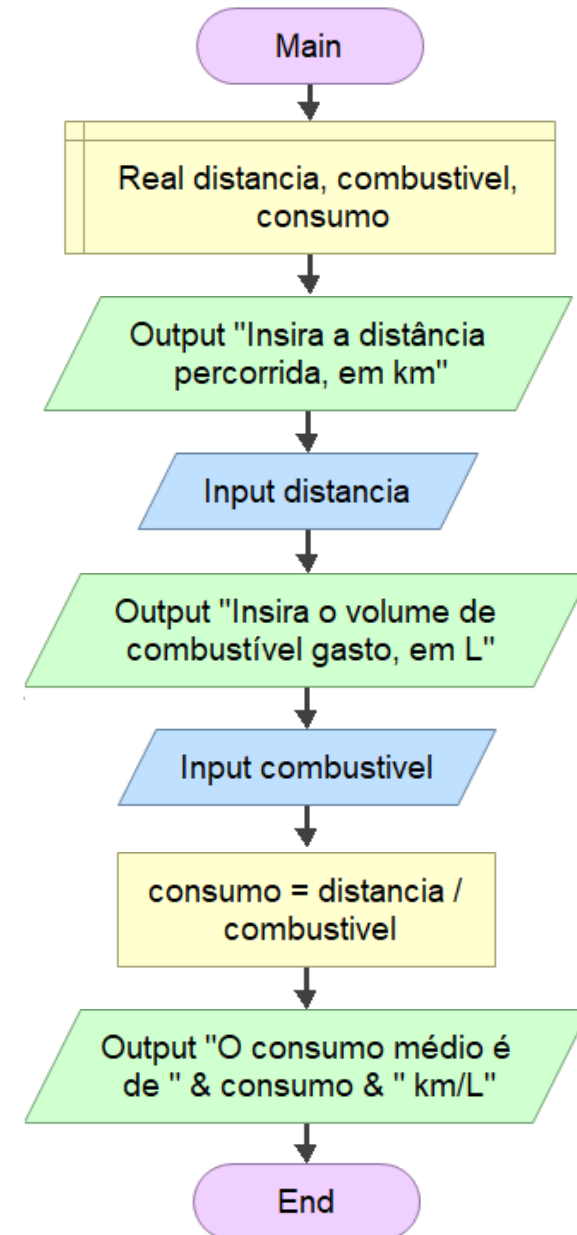
Exemplo:

650.5

50

O consumo médio é de 13.01 km/L

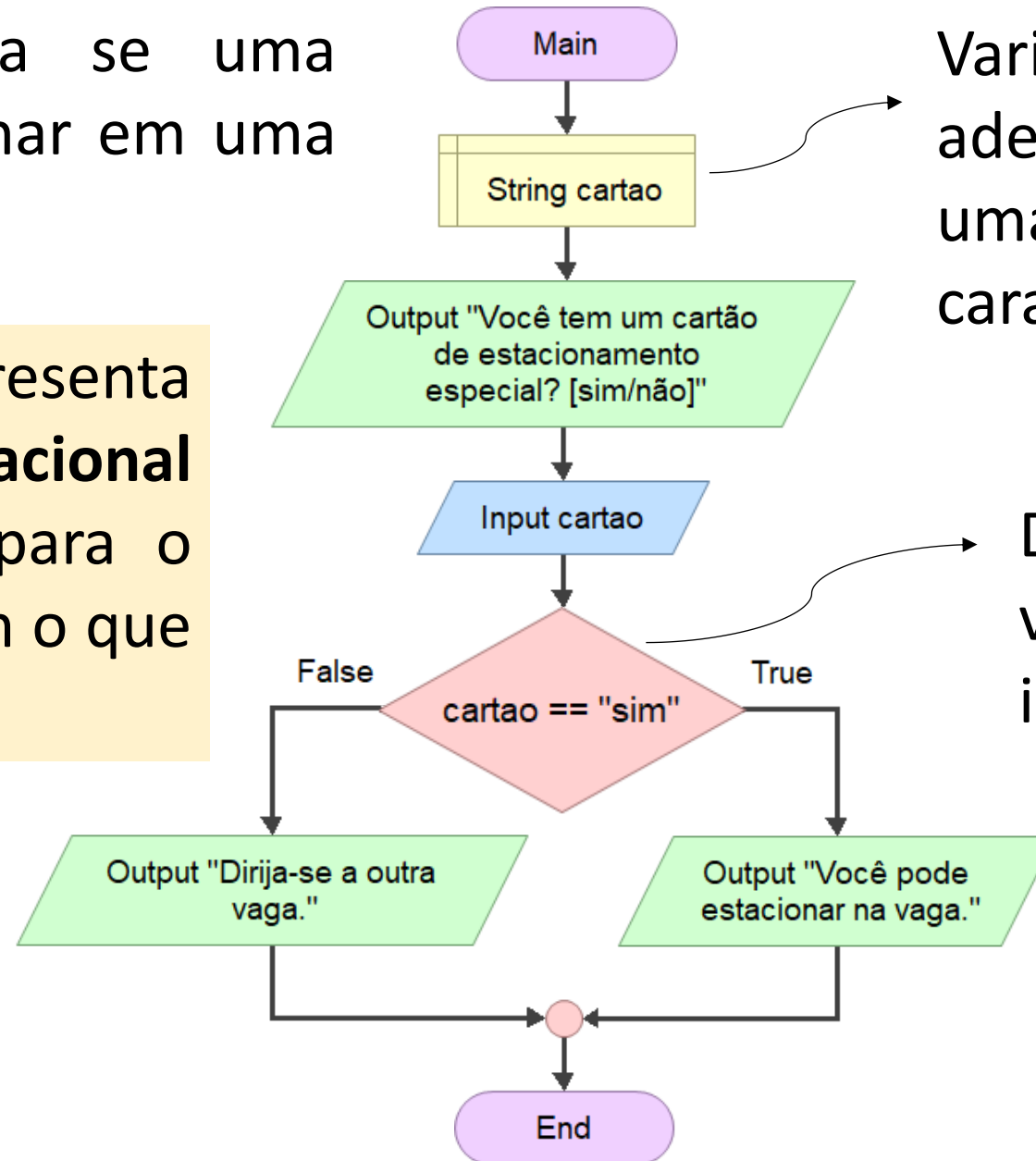
No Flowgorithm, o símbolo & representa um operador de **concatenação**. Com ele, podemos colocar, lado a lado, texto e valores de variáveis no mesmo comando de output.



Introdução à lógica de programação: exemplos de fluxogramas

Programa que testa se uma pessoa pode estacionar em uma vaga especial.

O símbolo **==** representa um **operador relacional “igual a”**. Ele compara o valor da variável com o que há à sua direita.



Variável `cartao`, adequada para abrigar uma sequência de caracteres (texto).

Decisão: verifica se o valor de `cartao` é igual a “sim”.

Introdução à linguagem Python: sobre a linguagem

Python é uma linguagem de programação de **alto nível**. Isso significa que ela possui um alto nível de abstração (utiliza comandos “parecidos” com a linguagem humana, para que não tenhamos que lidar com linguagem binária).



Como toda linguagem de programação, a linguagem Python é um meio de comunicação entre nós, humanos, e os circuitos computacionais. Por meio de seus comandos, conseguimos criar programas que enviam instruções ao *hardware*, fazendo com que ele realize o processamento que precisamos.

Desenvolvida pelo matemático holandês Guido van Rossum, Python teve sua primeira versão lançada em 1991. Sua última versão é a 3.14, lançada em 2025. CPython é a sua implementação principal, escrita em linguagem C. Python é muito versátil, e é amplamente utilizada em ciência de dados e em inteligência artificial.

Introdução à linguagem Python: compilador x interpretador

O código-fonte de um programa é um conjunto de palavras e símbolos que dita as instruções que o programa deve fazer. O código-fonte é escrito em linguagem próxima à dos humanos, mas não tem significado para o *hardware*.

Para fazer com que o nosso código seja entendido como um programa pelo computador, é preciso convertê-lo para linguagem de máquina. Há dois processos principais de conversão: **compilação** e **interpretação**.

Compilador

- As instruções são convertidas de uma só vez, antes da execução;
- É gerado um arquivo executável, que pode ser executado quantas vezes quisermos;
- A execução do programa costuma ser mais eficiente;
- C, C++, Go são estritamente compiladas.

Interpretador

- Cada instrução é convertida e executada antes da próxima conversão de instrução;
- Não é gerado um arquivo executável: a cada execução, é necessário um novo processo de conversão;
- A execução exige menos memória e pode acontecer de forma interativa;
- Python e JavaScript são interpretadas.

Introdução à linguagem Python: modos de programação

Podemos trabalhar com Python no modo interativo ou no modo *script*.

No **modo interativo**, “conversamos” com o interpretador por meio do *shell*, que responde aos comandos, linha a linha. Ótimo para entender e testar comandos.

```
>>> print("Hello World")
```

```
Hello World
```

```
>>> 4 + 2
```

```
6
```

```
>>>
```

No **modo *script***, escrevemos o código completo a ser executado, salvamos o arquivo com extensão `.py` e, depois, pedimos para o interpretador executá-lo. Tudo é executado, mas é exibido apenas o que foi solicitado.

Script.py:

```
1. print('Hello World')
```

```
2. 4 + 2
```

Saída:

```
Hello World
```

Introdução à linguagem Python: operadores aritméticos

Vamos testar os **operadores aritméticos** simulando o *shell* interativo do Python.

```
>>> 3 + 7
```

```
10
```

```
>>> 3 - 7
```

```
-4
```

```
>>> 3 * 2
```

```
6
```

```
>>> 5 / 2
```

```
2.5
```

```
>>> 4 / 2
```

```
2.0
```

```
>>> 15 // 2
```

```
7
```

```
>>> 15 % 2
```

```
1
```

```
>>> 2 ** 3
```

```
8
```

```
>>> 2 ** 4
```

```
16
```

```
>>> 3 + 2*3
```

```
9
```

```
>>> (3+2) * 3
```

```
15
```

```
>>> 5.2 + 2
```

```
7.2
```

```
>>> 2.5 + 8//2
```

```
6.5
```

```
>>> 2.5 + (8//2)
```

```
6.5
```

Operador	Operação	Retorna um valor do tipo
+	Adição	Real (float), inteiro (int)...
-	Subtração	
*	Multiplicação	
/	Divisão	Real (float)
//	Divisão inteira	Inteiro (int)
%	Resto da divisão	Inteiro (int)
**	Potenciação	Real (float), inteiro (int)...

Ordem de precedência:

1. Parênteses
2. Potenciação
3. Multiplicação e divisões (da esq. p/ direita)
4. Adição e subtração (da esquerda p/ direita)

Introdução à linguagem Python: operadores relacionais

Vamos testar **operadores relacionais**, que comparam valores entre si e retornam um resultado de tipo booleano (bool): ou **True** (verdadeiro) ou **False** (falso).

```
>>> 2 < 3
```

```
True
```

```
>>> 2 > 3
```

```
False
```

```
>>> 5 == 5
```

```
True
```

```
>>> 5 != 5
```

```
False
```

```
>>> 4 <= 5
```

```
True
```

```
>>> 4 <= 4
```

```
True
```

```
>>> 2+3 == 5
```

```
True
```

```
>>> 7-2 != 4+1
```

```
False
```

```
>>> 5 >= 10.9
```

```
False
```

```
>>> True == 1
```

```
True
```

```
>>> False == 0
```

```
True
```

```
>>> True + True
```

```
2
```

Operador	Operação	Retorna um valor do tipo
<	Menor que	Booleano (bool): ou True (1) ou False (0).
>	Maior que	
<=	Menor ou igual a	
>=	Maior ou igual a	
==	Igual a	
!=	Diferente de	

Ordem de precedência:

1. Parênteses
2. Operadores aritméticos
3. Operadores relacionais

Introdução à linguagem Python: operador de atribuição

O **operador de atribuição** (=) atribui um valor a uma variável, que é declarada em tempo de execução com um tipo compatível com o dado que se quer guardar.

```
>>> nome1 = 'Maria'
```

```
>>> nome1
```

```
'Maria'
```

```
>>> type(nome1)
```

```
<class 'str'>
```

```
>>> x = 2
```

```
>>> type(x)
```

```
<class 'int'>
```

```
>>> x = 34.56
```

```
>>> type(x)
```

```
<class 'float'>
```

```
>>> x = True
```

```
>>> type(x)
```

```
<class 'bool'>
```

```
>>> x + x + x
```

```
3
```

```
>>> r = s = 20
```

```
>>> r
```

```
20
```

```
>>> r = 30
```

```
>>> s
```

```
20
```

Uma **variável** é um espaço em memória que é tratado por um nome específico dentro do código. O valor e o tipo da variável podem mudar ao longo da execução.

O tipo `str` (*string*) designa conjuntos de caracteres alfanuméricos, como letras, textos, nomes, símbolos, etc.

Introdução à linguagem Python: função de saída `print()`

Considere o *script* a seguir.

```
1. print("Olá, mundo!")
```

Ao ser executado, o programa resulta em:

```
Olá, mundo!
```

`print()` exibe uma mensagem em tela - por isso, é uma função de saída (*output*).

`print()` é uma função embutida (*built-in*) do Python. Uma **função** é um conjunto de comandos que realizam uma tarefa específica. Desse modo, é um “pedaço” de código que pode ser invocado e executado diversas vezes.

Uma **função embutida** é uma função que já está automaticamente disponível para utilização (não há a necessidade de importá-la de algum módulo).

Os parênteses definem o início (“`(`”) e o fim (“`)`”) da lista de argumentos da função. **Argumento** é uma informação que pode ser passada para a função agir.

Introdução à linguagem Python: saída `print()` e entrada `input()`

Considere o *script* a seguir.

```
1. print("Digite seu nome: ")
2. nome = input()
3. print(f"Seu nome é {nome}")
4. print("Seja bem-vindo(a), " + nome)
```

`input()` é uma função embutida que coleta textos da entrada padrão da máquina (teclado).

Supondo que o usuário digitou "Ana", temos como resultado:

```
Digite seu nome:
Ana
Seu nome é Ana
Seja bem-vindo(a), Ana
```

Na linha 2, chamamos a função `input()`, que aguarda um texto ser digitado pelo usuário. Assim que é dada a entrada, o texto é atribuído à variável `nome`, que é declarada com tipo *string* (`str`) em tempo de execução, devido ao tipo de dado que se quer guardar (texto).

Introdução à linguagem Python: saída `print()` e entrada `input()`

Considere o *script* a seguir.

```
1. print("Digite seu nome: ")
2. nome = input()
3. print(f"Seu nome é {nome}")
4. print("Seja bem-vindo(a), " + nome)
```

Na linha 3, usamos uma **f-string**, que é uma *string* com a letra `f` no início e chaves `{}` para realizar a interpolação de expressões.

Interpolação é o processo de inserir valores diretamente dentro de uma *string*, usando *placeholders* (marcadores de posição).

Na linha 4, usamos o operador de concatenação `+`. Ele permite apenas a concatenação de *strings*. Variáveis de outros tipos precisam ser convertidas explicitamente usando a função `str()`.

Introdução à linguagem Python: comentários

Um **comentário** é um trecho de texto incluído no código para descrever alguma coisa, como por exemplo, o que aquele pedaço do programa faz. Os comentários não modificam o funcionamento do programa porque são ignorados pelo interpretador.

Comentário **por linha**: basta adicionar o caractere cerquilha (#) na frente da linha ou ao final de um comando. Tudo o que vier na linha depois de # será considerado comentário e ignorado pelo interpretador.

Comentário **por bloco**: basta adicionar aspas três vezes (""") ao início e ao fim do comentário, pegando quantas linhas forem necessárias.

Introdução à linguagem Python: exemplo de programa simples

Exemplo de programa que solicita um nº inteiro e exibe o dobro desse nº.

```
1.      """
2.      Programa que solicita um nº inteiro e
3.      exibe o dobro desse nº.
4.      """
5.
6.      num = int(input("Digite um nº inteiro: "))
7.      print(f"Dobro de {num} é {num * 2}")
8.
9.      # input também realiza saída de dados
10.     # input retorna tipo str por padrão
11.
```